

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Архитектура программных систем

Лабораторная работа №2

Выполнил:

Вильданов Ильнур Наилевич

Группа №Р3312

Преподаватель:

Перл Иван Андреевич

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

1. Задание	3
2. Strategy (GoF)	4
2.1. Описание шаблона Strategy	4
2.2. Сценарии применения	4
2.2.1. Расчет итоговой цены (скидки, промокоды, лояльность) в e-commerce.....	4
2.2.2. Выбор способа аутентификации (пароль, SSO, OAuth, сертификат) в корпоративном ПО	4
2.2.3. Ранжирование/сортировка в поиске (релевантность, цена, популярность, персонализация)	5
2.3. Итоговые ограничения Strategy	6
3. Factory Method (GoF).....	6
3.1. Описание шаблона Factory Method	6
3.2. Сценарии применения	6
3.2.1. Экспорт отчетов в разные форматы (PDF/Excel/CSV/JSON).....	6
3.2.2. Интеграция с несколькими платежными провайдерами.....	7
3.2.3. Парсинг входных файлов разных типов в ETL (XML/JSON/CSV).....	7
3.3. Итоговые ограничения Factory Method.....	8
4. Controller (GRASP).....	8
4.1. Описание принципа Controller	8
4.2. Сценарии применения	8
4.2.1. CheckoutController в интернет-магазине (оформление заказа)	8
4.2.2. BookingController в системе бронирований (переговорки/билеты/ресурсы).....	9
4.2.3. IncidentController в HelpDesk/ITSM (инциденты и заявки).....	9
4.3. Итоговые ограничения Controller.....	10
5. Information Expert (GRASP)	10
5.1. Описание принципа Information Expert	10
5.2. Сценарии применения	10
5.2.1. Подсчет суммы заказа и налогов (Order как эксперт по строкам заказа)	10
5.2.2. Определение прав доступа на действие (Role/Policy как эксперт)	11
5.2.3. Проверка конфликтов расписания (Schedule как эксперт по интервалам)	11
5.3. Итоговые ограничения Information Expert	12
6. Вывод	12

1. Задание

Из списка шаблонов проектирования GoF и GRASP выбрать 3-4 шаблона и для каждого из них придумать 2-3 сценария, для решения которых могут применены выбранные шаблоны.

Сделать предположение о возможных ограничениях, к которым можем привести использование шаблона в каждом описанном случае. Обязательно выбрать шаблоны из обоих списков.

2. Strategy (GoF)

2.1. Описание шаблона Strategy

Strategy (Стратегия) - поведенческий шаблон GoF, который позволяет выделить семейство алгоритмов, инкапсулировать каждый из них и сделать взаимозаменяемыми. Клиентский код работает через общий интерфейс и выбирает реализацию в зависимости от условий выполнения.

2.2. Сценарии применения

2.2.1. Расчет итоговой цены (скидки, промокоды, лояльность) в e-commerce

Сценарий: система продаж поддерживает несколько правил расчета цены: процентные/фиксированные скидки, промокоды, акции по сегментам клиентов, временные кампании. Правила регулярно меняются.

Применение: интерфейс PricingStrategy и реализации (PercentDiscount, FixedDiscount, PromoCodePricing, LoyaltyPricing). Расчет корзины выбирает стратегию по контексту (тип клиента, наличие промокода, период акции).

Ограничения:

- При большом количестве акций растет число стратегий - нужна конфигурация, каталог стратегий или правила выбора (policy).
- Риск дублирования условий (каждая стратегия повторяет проверки входных данных) - желательно выносить общие проверки отдельно.
- Усложняется тестирование: важно покрывать комбинации правил и крайние случаи (иначе возможны финансовые ошибки).

2.2.2. Выбор способа аутентификации (пароль, SSO, OAuth, сертификат) в корпоративном ПО

Сценарий: приложение должно поддерживать несколько способов входа, которые выбираются по политике безопасности и роли пользователя.

Применение: интерфейс AuthStrategy (authenticate/refresh/logout) и реализации PasswordAuth, SsoAuth, OAuthAuth, CertificateAuth. Модуль входа выбирает стратегию по настройкам организации.

Ограничения:

- Нужны правила приоритета и fallback (что делать, если SSO/IdP недоступен).
- Каждая стратегия имеет свои риски безопасности и требования к аудиту (MFA, срок жизни токенов, логирование).
- Поддержка множества протоколов повышает стоимость сопровождения и интеграционного тестирования.

2.2.3. Ранжирование/сортировка в поиске (релевантность, цена, популярность, персонализация)

Сценарий: поисковая выдача должна перестраиваться по различным критериям, а также отличаться для разных разделов/типов пользователей.

Применение: RankingStrategy/SortStrategy и реализации RelevanceRanking, PriceSort, PopularityRanking, PersonalizedRanking. Поисковый модуль выбирает стратегию по параметрам запроса.

Ограничения:

- Персонализированные стратегии требуют дополнительных данных и вычислений (latency, кэширование, нагрузка).
- Сложнее отлаживать и объяснять выдачу пользователю - нужен лог причин ранжирования и воспроизводимость.
- Если логика выбора стратегии «размазана» по коду, появляется технический долг - выбор лучше централизовать.

2.3. Итоговые ограничения Strategy

Strategy повышает гибкость и упрощает изменение алгоритмов, но увеличивает число сущностей и требует аккуратного механизма выбора стратегии и качественного тестового покрытия.

3. Factory Method (GoF)

3.1. Описание шаблона Factory Method

Factory Method (Фабричный метод) - порождающий шаблон GoF, который определяет интерфейс создания объекта, позволяя подклассам/фабрике решать, какой класс инстанцировать. Используется для отделения логики создания от логики использования.

3.2. Сценарии применения

3.2.1. Экспорт отчетов в разные форматы (PDF/Excel/CSV/JSON)

Сценарий: отчетная подсистема должна поддерживать несколько форматов экспорта, набор форматов расширяется со временем.

Применение: ExporterFactory создает

PdfExporter/XlsxExporter/CsvExporter/JsonExporter по параметру формата. Код генерации отчета работает с общим интерфейсом Exporter.

Ограничения:

- Фабрика может разрастись в «switch-комбайн» - лучше использовать регистрацию реализаций или DI-контейнер.
- Форматы имеют разные ограничения (верстка PDF vs плоский CSV) - интерфейс экспортера нужно проектировать осознанно.
- Нужны тесты корректности форматов и контроль зависимостей библиотек (версии, лицензии).

3.2.2. Интеграция с несколькими платежными провайдерами

Сценарий: система должна проводить платежи через разных провайдеров (география, резервирование, требования бизнеса).

Применение: PaymentGatewayFactory возвращает адаптер нужного провайдера (Stripe/Adyen/локальный). Доменный код вызывает единый интерфейс PaymentGateway.

Ограничения:

- Слишком общий интерфейс может «протекать» из-за различий провайдеров (3-DS, статусы, частичные возвраты).
- Нужны контрактные тесты и песочницы провайдеров - иначе ошибки выявляются только на реальных платежах.
- Fallback-выбор по доступности требует мониторинга и политики переключения - это выходит за рамки «простой фабрики».

3.2.3. Парсинг входных файлов разных типов в ETL (XML/JSON/CSV)

Сценарий: ETL-процесс получает файлы разных форматов и должен привести их к единому внутреннему представлению.

Применение: ParserFactory определяет тип и создает

XmlParser/JsonParser/CsvParser. Дальше пайплайн работает с ParsedData.

Ограничения:

- Определение типа может быть неоднозначным (ошибочные расширения, разные версии схем) - нужны валидация и проверки.
- Разные парсеры имеют разные требования к памяти/скорости (поточковый vs в память).
- Фабрика не решает проблему совместимости схем - потребуется слой нормализации/маппинга.

3.3. Итоговые ограничения Factory Method

Factory Method улучшает расширяемость и снижает связанность, но важно не превращать фабрику в монолитный набор условий и поддерживать устойчивый интерфейс создаваемых объектов.

4. Controller (GRASP)

4.1. Описание принципа Controller

Controller - принцип GRASP, предлагающий назначать обработку системных событий (запросов от UI или внешних клиентов) специальному объекту-контроллеру. Контроллер координирует сценарий и делегирует бизнес-логику доменным объектам/сервисам.

4.2. Сценарии применения

4.2.1. CheckoutController в интернет-магазине (оформление заказа)

Сценарий: оформление заказа включает валидацию корзины, расчет доставки, резервирование товаров, оплату и подтверждение.

Применение: CheckoutController принимает команду и вызывает CartService/InventoryService/DeliveryService/PaymentService, возвращая статус и ошибки. Бизнес-правила остаются в сервисах, контроллер - координирующий.

Ограничения:

- Риск «God Controller» - если контроллер хранит бизнес-правила, он становится крупным и плохо тестируемым.
- Нужна идемпотентность и защита от повторных действий (двойная оплата).
- Требуется трассировка между сервисами (correlation id), иначе сложно расследовать сбои.

4.2.2. BookingController в системе бронирований (переговорки/билеты/ресурсы)

Сценарий: создание/перенос/отмена брони во временных слотах, конфликты расписания и ограничения по ролям.

Применение: BookingController - единая точка входа для команд create/cancel/reschedule; делегирует проверки SlotService и PolicyService.

Ограничения:

- Высокая конкурентность требует механизма предотвращения двойного бронирования (версии, блокировки, транзакции).
- Контроллер должен выдавать понятные ошибки конфликтов, иначе пользователи будут «ломать» процесс.
- Если контроллер зависит от множества сервисов, растет связанность - может потребоваться декомпозиция по сценариям.

4.2.3. IncidentController в HelpDesk/ITSM (инциденты и заявки)

Сценарий: создание инцидента, изменение статусов, эскалации, назначение исполнителя, аудит.

Применение: IncidentController проверяет права, вызывает WorkflowService и NotificationService, записывает аудит.

Ограничения:

- Контроллер может разрастись из-за множества операций - важно держать его «тонким» и делегирующим.
- Ошибки в проверке прав приводят к уязвимостям (неправомерное изменение статуса).
- Жесткая привязка к конкретному workflow - при изменении процессов потребуется аккуратный рефакторинг.

4.3. Итоговые ограничения Controller

Controller повышает ясность границы системы и концентрирует обработку запросов, но требует дисциплины: контроллеры должны быть координационными, а не содержать бизнес-логику.

5. Information Expert (GRASP)

5.1. Описание принципа Information Expert

Information Expert (Информационный эксперт) - принцип GRASP: ответственность следует назначать тому объекту, который обладает необходимой информацией для ее выполнения. Это помогает строить модель предметной области естественно и снижает ненужные зависимости.

5.2. Сценарии применения

5.2.1. Подсчет суммы заказа и налогов (Order как эксперт по строкам заказа)

Сценарий: нужно корректно вычислять итоговую сумму заказа (subtotal, taxes, delivery, total) на основе позиций и правил.

Применение: ответственность расчета subtotal/total передается объекту Order (или Cart), потому что он содержит OrderLine и знает их количества, цены и параметры.

Ограничения:

- Риск перегрузить доменный объект техническими деталями (например, форматирование отчета/валюты) - это лучше вынести в отдельный слой.
- Если расчеты требуют данных из внешних источников (курсы валют, налоговые ставки), объект-эксперт может начать зависеть от инфраструктуры.
- При сложных правилах возможно разрастание модели - иногда нужна комбинация с Strategy или отдельным доменным сервисом.

5.2.2. Определение прав доступа на действие (Role/Policy как эксперт)

Сценарий: система должна решать, может ли пользователь выполнять конкретное действие (создать/изменить/удалить/одобрить).

Применение: объект Role/PermissionPolicy хранит набор разрешений и становится экспертом для ответа на вопрос `canPerform(action, resource)`. Контроллеры лишь спрашивают эксперта, а не «знают» правила сами.

Ограничения:

- Если правила зависят от множества факторов (контекст, время, атрибуты ресурса), эксперт может стать слишком сложным.
- Нужно избегать дублирования правил в разных местах - все проверки должны проходить через единый механизм.
- При большом количестве правил потребуется аудит/логирование решений (почему доступ разрешен/запрещен).

5.2.3. Проверка конфликтов расписания (Schedule как эксперт по интервалам)

Сценарий: нужно определить, пересекается ли новый временной интервал с существующими событиями и можно ли добавить запись.

Применение: объект Schedule (или Calendar) содержит события/интервалы и является экспертом для операций `hasConflict(newInterval)`, `addEvent(event)`.

Ограничения:

- Если события хранятся удаленно и выборка дорогая, эксперту потребуется оптимизация (индексы, кэширование), иначе падает производительность.
- Сложные правила (повторяющиеся события, таймзоны) увеличивают сложность эксперта и тестов.
- При высокой конкурентности нужно обеспечить корректность при параллельных изменениях (`optimistic lock/transaction`).

5.3. Итоговые ограничения Information Expert

Information Expert делает модель естественной и уменьшает связанность, но важно не смешивать доменные обязанности с инфраструктурой. При сложных вычислениях или зависимостях от внешних данных может потребоваться выделение доменных сервисов или комбинация с другими паттернами.

6. Вывод

В рамках лабораторной работы были рассмотрены 4 шаблона/принципа из списков GoF и GRASP: Strategy и Factory Method (GoF), Controller и Information Expert (GRASP). Для каждого предложены по 3 сценария применения и описаны возможные ограничения, к которым может привести использование паттерна в каждом случае.